НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

# Протокол лабораторної роботи №1
## «розгортання середовища та порівняльний аналіз мовних моделей»
## з дисципліни: «Обчислювальні методи»

Виконав: студент групи AI-245
Штумбенко Р. О.

Перевірив: ас. Баранов О.М.

Одеса – 2026

## Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт /api/generate. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

| Показник | qwen3:4b | llama3.2:3b |
|---|---|---|
| response | Киев | Київ |
| eval_count | 144 | 5 |
| Час, с | 16,48 | 4,95 |
| Токенів/с | 22,29 | 48,08 |

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

| Показник | qwen3:4b | llama3.2:3b |
|---|---|---|
| Правильних із 5 | 5 / 5 | 1 / 5 |
| eval_count | 1465,4 | 198,6 |
| Час, с | 76,21 | 6,42 |

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

**Відповіді на питання (крок 6)**

1. Куди пішли додаткові токени в міркувальної моделі?

Відповідь: Міркувальні моделі (qwen3:4b) навчені формувати внутрішній ланцюжок думок (Chain-of-Thought) перед виведенням остаточної відповіді. Тому більшість зі 144 згенерованих токенів була витрачена на прихований покроковий процес "обдумування", тоді як сам результат склав лише одне слово.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Такі моделі дають значну перевагу у складних логічних, математичних чи аналітичних завданнях, що підтверджується абсолютною точністю (5/5) у задачі на міркування. Натомість для простих фактичних запитів їх використання невиправдане через марні витрати часу (16,48 с проти 4,95 с) та обчислювальних ресурсів на непотрібні логічні кроки.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Інструктивні моделі (Supervised Fine-Tuning) навчаються давати пряму і швидку відповідь у стилі звичайної розмови. Міркувальні моделі додатково проходять спеціальне навчання (наприклад, через навчання з підкріпленням), де їх стимулюють генерувати розгорнутий, правильний покроковий алгоритм вирішення перед тим, як надати фінальний висновок.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Сама по собі кількість токенів не є прямим показником «розуму», а лише відображає підхід моделі до формування проміжних кроків. На простих питаннях зайві токени (144 проти 5) свідчать радше про обчислювальну надмірність, але на складних задачах саме цей довгий покроковий механізм (1465,4 токенів) забезпечує вищу результативність (5/5 правильних).

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: Різниця у швидкості (22,29 проти 48,08 токенів за секунду) зумовлена насамперед розміром та оптимізацією моделей. qwen3:4b має 4 мільярди параметрів, що вимагає більше пам'яті та обчислень на кожен згенерований токен, тоді як менша llama3.2:3b працює значно швидше на тому ж обладнанні.

**Висновок за завданням 1**

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати відрізняються у 28,8 разів. На задачі про ручку й зошит ситуація протилежна – саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної

відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе 609,7 с проти 51,4 с, що робить міркувальну модель непридатною для налагодження.

### Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Таблиця 3 – Вплив system повідомлення

| Модель | System | Довжина відповіді, символів | Мова відповіді | Кількість речень | Інструкцію виконано? |
|---|---|---|---|---|---|
| llama3.2:3b | лаконічний асистент | 2363 | суржик з українською та російською зі вставками іспанських, французьких, чеських слів | 17 | Ні |
| llama3.2:3b | українською, 2 речення | 253 | українська з англійською вставкою та неологізмами | 2 | Частково |
| qwen3:4b | лаконічний асистент | 564 | українська з англомовними IT-термінами | 9 | Так |
| qwen3:4b | українською, 2 речення | 395 | українська | 2 | Так |

Таблиця 4 – Витрати токенів

| Модель | System | prompt_tokens | completion_tokens | total_tokens |
|---|---|---|---|---|
| llama3.2:3b | лаконічний асистент | 60 | 680 | 740 |
| llama3.2:3b | українською, 2 речення | 64 | 68 | 132 |
| qwen3:4b | лаконічний асистент | 69 | 1630 | 1699 |
| qwen3:4b | українською, 2 речення | 78 | 805 | 883 |

**Відповіді на питання (крок 6)**

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Обидві моделі вклалися в ліміт із 2 речень, проте qwen3:4b впоралася значно точніше й змістовніше, сформулювавши два граматично коректні та інформативні речення (395 символів) без втрати змісту. Модель llama3.2:3b формально теж видала 2 речення (253 символи), але суттєво спотворила їхній синтаксис і лексику («на contrary», «взаємосполуччя»).

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Так, модель llama3.2:3b систематично порушувала мовний режим: при першому промпті генерувала суржик із російськими («как набір записей», «храняться»), іспанськими («mediante»), французькими («valeur», «nécessyють») та чеськими («vyžadyють») словами, а при прямій вказівці писати лише українською вставила англійське слово «contrary». Модель qwen3:4b в обох випадках відповідала чистою українською мовою (з доречними IT-термінами SQL, joins, ACID, BASE, JSON/BSON).

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Показник prompt_tokens зріс для обох моделей (з 60 до 64 у llama3.2:3b та з 69 до 78 у qwen3:4b), оскільки друга інструкція («Відповідай лише українською, максимум двома реченнями.») довша за кількістю слів і символів, ніж перша («Ти лаконічний технічний асистент.»). Додатковий приріст зумовлений особливостями токенізації кириличного тексту, який розбивається на більшу кількість субтокенів порівняно з латиницею.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Модель qwen3:4b є міркувальною і перед формуванням відповіді будує розлогий прихований ланцюжок роздумів. Показник completion_tokens фіксує всі згенеровані токени включно з цим внутрішнім монологом (805 токенів для 2 речень), тоді як звичайна інструктивна модель llama3.2:3b генерує лише видимий текст (68 токенів).

**Відповіді на питання (крок 7)**

1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь: Міркувальна модель qwen3:4b краще дотримується інструкцій (наприклад, чітко видає 2 речення), оскільки використовує свої можливості для планування та контролю обмежень під час генерації. Натомість менша інструктивна llama3.2:3b часто ігнорує системний промпт, генеруючи 17 речень замість лаконічної відповіді, через слабшу здатність утримувати складні правила в контексті.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Відповідь: System-повідомлення задає базові поведінкові рамки на рівні архітектури, тому для моделі воно має слугувати пріоритетним правилом. Проте на практиці пріоритетність залежить від моделі: qwen3:4b стабільно виконує системні настанови, тоді як llama3.2:3b може їх повністю або частково проігнорувати.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь: Міркувальні моделі перед виведенням тексту завжди генерують прихований проміжний ланцюжок думок. Через це для видачі видимої відповіді обсягом у 2 речення (395 символів) qwen3:4b витрачає аж 805 completion-токенів на внутрішні розмірковування, тоді як llama3.2:3b витрачає лише 68.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Відповідь: За замовчуванням якісно налаштована модель повинна віддати пріоритет system-повідомленню як базовому обмежувачу. Проте, як свідчать дані, малі інструктивні моделі (llama3.2:3b) легко втрачають фокус і можуть повністю провалити системну настанову (результат виконання «Ні»). Міркувальні моделі більш стійкі та з високою ймовірністю збережуть системні рамки.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: Через менший обсяг параметрів малим моделям (як llama3.2:3b) важко одночасно утримувати логіку інструкції та генерувати текст не основною для них мовою, що призводить до суржику, іноземних вставок та неологізмів. Модель qwen3:4b уникає цього, оскільки виділяє додаткові токени на внутрішнє планування, чітко зберігаючи задану українську мову (іноді з профільними IT-термінами).

**Висновок за завданням 2**

Експеримент довів, що system-повідомлення є зручним способом задати тон, мову та обсяг відповіді без зміни коду, проте воно діє лише як м'який статистичний вектор, а не жорстке програмне обмеження. Мала інструктивна модель llama3.2:3b не впоралася з роллю «лаконічного асистента», згенерувавши розтягнутий текст на 17 речень (2363 символи) із суміші чотирьох мов, а при прямому обмеженні двома реченнями все одно припустилася англомовної вставки. Натомість міркувальна qwen3:4b завдяки попередньому плануванню у внутрішньому монолозі точно

виконала обидві інструкції, але заплатила за це прихованою генерацією 1630 та 805 токенів виводу. Отже, самої лише текстової інструкції в system недостатньо для гарантованого отримання потрібного формату від малих моделей: якщо модель ігнорує системні вказівки, необхідно застосовувати жорстку програмну валідацію на боці коду, обмежувати генерацію параметром max_tokens або використовувати більш містку модель.

### Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5 – Порівняння локальної та хмарної моделі

| Показник | Локальна | Хмарна |
|---|---|---|
| Назва моделі | llama3.2:3b | gemini-3.5-flash-lite |
| Час відповіді, с | 13,48 | 6,51 |
| completion_tokens | 140 | 349 |
| Довжина відповіді, символів | 489 | 1025 |
| Мова відповіді | суржик із російськими та англійськими словами | літературна українська |
| Фактичних помилок у відповіді | неточне визначення NoSQL як «інформативної моделі» та граматичний брак у першому ж реченні | 0 |

**Відповіді на питання (крок 5)**

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Завдяки єдиному стандарту інтерфейсу OpenAI API та винесенню налаштувань (base_url, api_key, model) у словник PROVIDERS у файлі providers.py, у самій логіці

виклику client.chat.completions.create() не довелося змінювати жодного рядка — перемикання відбувається зміною 1 аргументу у функції make_client(name) (або заміною 2 параметрів ініціалізації клієнта: адреси сервера та ключа).

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь хмарної моделі gemini-3.5-flash-lite є значно повнішою, точнішою та краще структурованою (1025 символів проти 489). Хмарна модель чітко порівняла обидва типи БД за трьома критеріями («Структура», «Гнучкість», «Для чого»), навела реальні приклади СУБД (PostgreSQL, MySQL проти MongoDB, CouchDB), згадала транзакції ACID та формат JSON, тоді як локальна llama3.2:3b обмежилася загальними тавтологічними фразами про «велику кількість даних» без жодного конкретного прикладу.

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Так, локальна модель llama3.2:3b припустилася як термінологічних, так і грубих мовних помилок. Вона некоректно назвала документоорієнтовану БД (NoSQL) «інформативною моделлю», згенерувала граматично зламане перше речення («Реляційною базою даних створюється за допомогою реляційної моделі...»), використала русизми («модели», «ключей», «строгою») та вставила англійське слово «flexibility» посеред українського речення.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Хмарна модель відповіла більш ніж удвічі швидше (6.51 с проти 13.48 с), згенерувавши при цьому у 2.5 раза більше токенів (349 проти 140). Це відбувається за рахунок того, що хмарний провайдер виконує обчислення на високопродуктивних серверних прискорювачах (промислових GPU/TPU) із величезною пропускною здатністю пам'яті, швидкодія яких із великим запасом перекриває мережеву затримку передачі пакета через інтернет.

**Відповіді на питання (крок 6)**

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: Формат запитів OpenAI став індустріальним стандартом завдяки тому, що компанія першою вийшла на масовий ринок і створила навколо свого API величезну екосистему інструментів та бібліотек. Підтримка цього стандарту іншими провайдерами, зокрема Ollama, дозволяє розробникам швидко перемикатися між різними моделями, не переписуючи логіку застосунків.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Розмір моделі має вирішальний вплив на якість генерації та розуміння контексту: хмарна модель gemini-3.5-flash-lite надала відповідь літературною українською мовою взагалі без фактичних помилок. Натомість менша локальна модель llama3.2:3b використала суржик із російськими та англійськими словами, а також припустилася помилки, назвавши NoSQL «інформативною моделлю».

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Хмарна модель впоралася із завданням за 6,51 секунди, згенерувавши 349 токенів, тоді як локальна витратила 13,48 секунди лише на 140 токенів. Ця різниця пояснюється тим, що хмарні моделі працюють на спеціалізованих кластерах із промисловими GPU/TPU, потужність яких повністю компенсує будь-які мережеві затримки порівняно з обмеженими ресурсами звичайного комп'ютера.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Передача даних через хмарні API створює ризик того, що конфіденційна корпоративна інформація або персональні дані користувачів можуть бути перехоплені чи використані провайдером для донавчання своїх моделей. Для мінімізації цих ризиків комерційні проєкти укладають Enterprise-угоди, що забороняють використання їхніх даних (zero-data retention), або розгортають власні ізольовані локальні моделі.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Rate limit – це встановлене провайдером обмеження на кількість запитів до API за певну одиницю часу (наприклад, за хвилину) для уникнення перевантаження серверів у безкоштовних тарифах. Розробники адаптуються до цього, впроваджуючи алгоритми автоматичної затримки перед повторною спробою (exponential backoff) або розподіляючи запити між кількома ключами чи акаунтами.

**Висновок за завданням 3**

Експеримент наочно продемонстрував головну архітектурну перевагу сучасної розробки ШІ-систем: завдяки уніфікованому OpenAI-сумісному інтерфейсу один і той самий код однаково працює як із локальною моделлю на власному ПК, так і з потужним хмарним кластером. Водночас порівняння результатів виявило колосальний розрив у продуктивності та якості: хмарна gemini-3.5-flash-lite не лише згенерувала вдвічі довшу, бездоганно структуровану відповідь літературною українською мовою без жодної помилки, а й зробила це вдвічі швидше (6.51 с проти 13.48 с у локальної llama3.2:3b). На практиці я обиратиму локальну модель, попри її меншу потужність і мовні огріхи, у випадках роботи з конфіденційними персональними чи корпоративними даними, які суворо заборонено передавати за межі локального контуру, для роботи в офлайн-середовищі без доступу до інтернету, а також для безкоштовного тестування та відлагодження архітектури коду без ризику вичерпати rate limit. Натомість хмарну модель доцільно обирати для всіх клієнтських і продуктових задач, де критичними є висока швидкість відгуку, глибоке розуміння контексту, фактологічна точність та якісна українська мова, а передача тексту на зовнішній сервер не порушує вимог безпеки.

**Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту**

Таблиця 6 – Порівняння локальної та хмарної моделі

| Розміщення | Модель | Промпт | TTFT, с | Всього, с | Симв./с |
|---|---|---|---|---|---|
| локальна | llama3.2:3b | короткий | 2,149 | 2,819 | 10,7 |
| локальна | llama3.2:3b | середній | 2,146 | 4,595 | 69,6 |
| локальна | llama3.2:3b | довгий | 2,155 | 18,202 | 115,3 |
| хмарна | gemini-3.5-flash-lite | короткий | 0,764 | 0,811 | 75,2 |
| хмарна | gemini-3.5-flash-lite | середній | 0,724 | 1,163 | 359,3 |
| хмарна | gemini-3.5-flash-lite | довгий | 0,842 | 3,366 | 776,2 |

Таблиця 7 – Холодний і теплий старт локальної моделі

| Показник | Холодний старт | Теплий старт |
|---|---|---|
| TTFT, с | 2,135 | 2,149 |
| Всього, с | 2,604 | 2,819 |

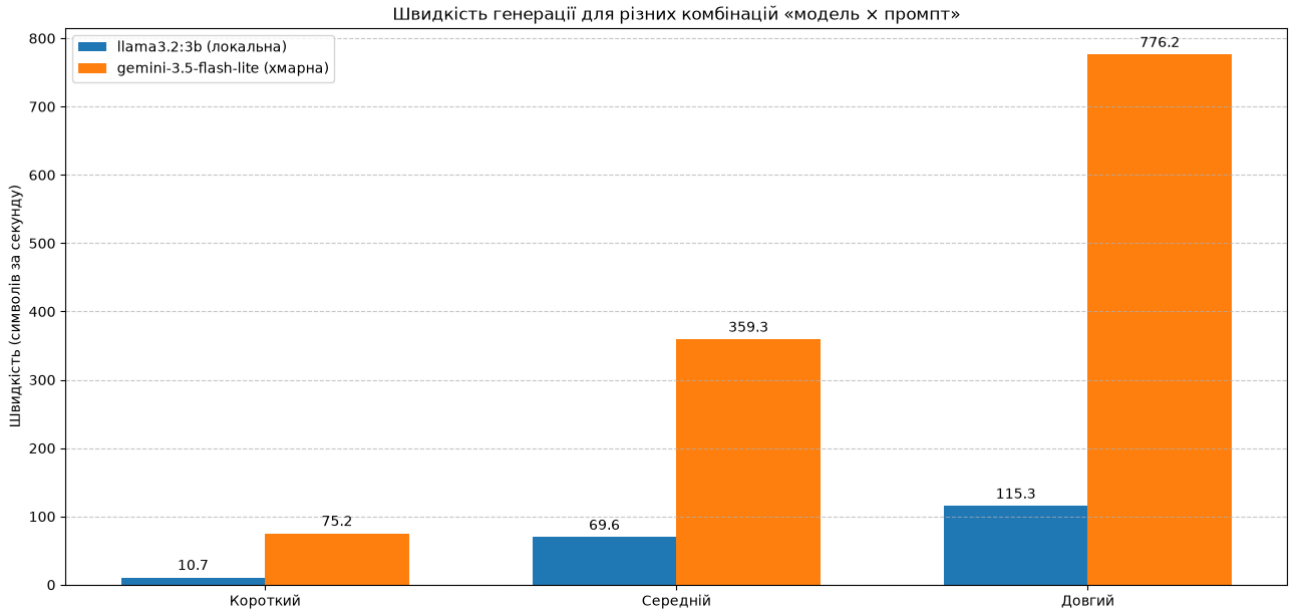



Рисунок 1 – Швидкість генерації для різних комбінацій «модель × промпт»

**Відповіді на питання (крок 6)**

1. У якої моделі кращий TTFT і чому саме в неї?

Кращий (менший) показник TTFT продемонструвала хмарна модель gemini-3.5-flash-lite (0.724–0.842 с проти 2.146–2.155 с у локальної llama3.2:3b). Попри витрати часу на передачу запиту мережею, хмарний кластер Google виконує фазу початкової обробки промпту (prefill) на спеціалізованих серверних прискорювачах майже втричі швидше, ніж локальне споживче обладнання.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Так, у фінальній серії вимірів повністю збігається: хмарна модель gemini-3.5-flash-lite випередила локальну як за швидкістю появи першого токена (TTFT ~0.76 с проти ~2.15 с), так і за швидкістю потокової генерації тексту на всіх типах промптів (від 75.2 до 776.2 симв./с проти 10.7–115.3 симв./с).

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

З переходом від короткого промпту до довгого розрахункова швидкість генерації суттєво зросла в обох моделей (у локальної – в 10.8 раза, з 10.7 до 115.3 симв./с; у хмарної – в 10.3 раза, з 75.2 до 776.2 симв./с), оскільки на короткій відповіді більшу

частину загального часу total_s з'їдає стартова затримка TTFT. При цьому хмарна модель стабільно утримувала перевагу у 5.2–7.0 разів за швидкістю генерації, а загальний час виконання довгого запиту в хмарі склав лише 3.366 с проти 18.202 с локально.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Повтори можуть суттєво різнитися, особливо для хмарного API: наприклад, у ранковій серії замірів через тимчасове навантаження на сервери чи мережу TTFT хмарної моделі стрибав до 5.044–5.954 с, а швидкість на середньому промпті просідала до 63.0 симв./с, тоді як у вечірній серії TTFT стабілізувався на рівні 0.724–0.842 с при швидкості 359.3–776.2 симв./с. Тому одного запуску категорично недостатньо – лише кілька повторів із обчисленням медіани дозволяють відсіяти випадкові мережеві та системні викиди.

5. Чому холодний старт повільніший за теплий?

Теоретично холодний старт повільніший тому, що під час першого запиту операційна система та сервіс Ollama мають зчитати гігабайтні файли ваг моделі з диска (SSD/HDD) та розмістити їх в оперативній або відеопам'яті, тоді як при теплому старті модель уже знаходиться у швидкій пам'яті. У моєму експерименті показники холодного (TTFT 2.135 с, всього 2.604 с) і теплого (TTFT 2.149 с, всього 2.819 с) стартів виявилися практично однаковими через малий розмір моделі llama3.2:3b (~2 ГБ), миттєве зчитування з SSD та агресивне кешування файлів в оперативній пам'яті ОС Windows.

**Відповіді на питання (крок 7)**

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: Під час TTFT (часу до першого токена) відбувається етап обробки вхідного запиту (префіл): токенізація, завантаження контексту та масивні паралельні обчислення для підготовки першого слова. Цей показник вимірюють

окремо, оскільки він відображає стартову затримку до початку видимої реакції системи, що є найважливішим фактором сприйняття швидкості користувачем.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: У хмарних моделях до TTFT додаються фізичні мережеві затримки (передача даних туди й назад) та можливий час очікування в серверній черзі, чого немає на локальному комп'ютері. Проте після старту генерації потужні хмарні GPU обробляють текст набагато швидше, хоча в наведених тестах хмарна gemini-3.5-flash-lite випередила локальну модель і за загальною швидкістю (до 776,2 симв./с), і за показником TTFT (0,7–0,8 с проти приблизно 2,1 с).

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: Холодний старт виникає при першому зверненні до моделі, коли її гігабайти ваг ще не завантажені у швидку відеопам'ять (VRAM) і зчитуються з накопичувача (наприклад, SSD), що зазвичай потребує додаткового часу. Незважаючи на це загальне правило, на наведених результатах тестування показники холодного та теплого стартів виявилися майже однаковими (загальний час 2,604 с та 2,819 с відповідно), що може свідчити про системне кешування перед вимірюванням.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Одиничний запуск часто спотворюється випадковими тимчасовими факторами: раптовим фоновим навантаженням системи, стрибками пінгу, або роботою фонових процесів операційної системи. Використання медіани з кількох запусків відкидає такі екстремальні відхилення і дає найбільш об'єктивну оцінку реальної продуктивності.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Обробка вхідного запиту виконується матрично та паралельно, оскільки всі слова промпту вже відомі моделі одночасно. Генерація відповіді є

авторегресивною: кожен новий токен обчислюється послідовно на основі всіх попередніх, що неможливо розпаралелити і вимагає постійного перерахунку, що добре видно по збільшенню загального часу роботи (до 18,202 с) при стабільному TTFT (2,155 с) для довгих запитів.

**Висновок за завданням 4**

Експеримент показав необхідність чіткого розмежування двох метрик швидкодії – стартової затримки (TTFT) та пропускної здатності генерації (симв./с або ток./с). Для інтерактивного чат-бота, де користувач у реальному часі дивиться на екран у режимі стрімінгу (stream=True), найважливішим показником є низький TTFT: якщо перші слова з'являються миттєво (як 0.72–0.84 с у gemini-3.5-flash-lite), інтерфейс сприймається як швидкий та чутливий, а подальший текст користувач читає поступово в міру його друкування. Натомість для фонової пакетної обробки великих обсягів тексту (масової класифікації документів, нічної генерації звітів чи парсингу логів без участі людини) стартова затримка TTFT не грає суттєвої ролі, а вирішальною стає висока швидкість генерації (симв./с), оскільки саме вона визначає загальний час виконання всього масиву задач (3.366 с у хмарної моделі при 776.2 симв./с проти 18.202 с у локальної при 115.3 симв./с на одному довгому запиті).

## Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 8 – Оцінка якості за рубрикою

| Завдання | llama3.2:3b | qwen3:4b | Хмарна модель |
|---|---|---|---|
| 1. Факт | 0 | 0 | 2 |
| 2. Формат | 2 | 2 | 2 |
| 3. Стиснення | 0 | 2 | 2 |
| 4. Код | 1 | 2 | 1 |
| 5. Логіка | 0 | 1 | 2 |

| | | | |
|---|---|---|---|
| 6. Українська | 0 | 1 | 2 |
| Разом /12 | 3 | 8 | 11 |

Таблиця 9 – Розбіжність оцінок між двома прогонами (так / ні)

| Завдання | llama3.2:3b | qwen3:4b | Хмарна модель |
|---|---|---|---|
| 1. Факт | так | так | ні |
| 2. Формат | ні | ні | ні |
| 3. Стиснення | так | ні | ні |
| 4. Код | так | ні | ні |
| 5. Логіка | так | ні | ні |
| 6. Українська | так | ні | ні |

**Відповіді на питання (крок 6)**

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Найбільше балів набрала хмарна модель gemini-3.5-flash-lite (11 із 12), далі йде локальна міркувальна qwen3:4b (8 із 12), а найменше отримала локальна інструктивна llama3.2:3b (3 із 12). Основна різниця між хмарною та локальними моделями виникла на регіональному факті (Завдання 1, де обидві локальні моделі галюцинували), багатокроковій логіці (Завдання 5) та якості української мови (Завдання 6), тоді як у генерації коду (Завдання 4) перемогла qwen3:4b, оскільки хмарна модель через відсутність у промпті назви мови написала предикат на Prolog у Markdown-обгортці.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Найбільша розбіжність між прогонами спостерігалася у llama3.2:3b (відповіді суттєво різнилися у 5 із 6 завдань: у Завданні 1 видала роки 1867 і 1869, у Завданні 3 – три речення проти одного без чисел, у Завданні 4 у другому прогоні загубила

підкреслення у назві функції tripcost, у Завданні 5 видала абсолютно різні хибні розрахунки) та у qwen3:4b на Завданні 1 (вигадала різні роки 1930 і 1924). Це свідчить про низьку надійність та високу стохастичну нестабільність малих моделей (особливо 3B) на задачах, де їм бракує знань або обчислювальної глибини, тоді як хмарна модель продемонструвала повну стабільність між прогонами.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Так, усі три моделі (llama3.2:3b, qwen3:4b та gemini-3.5-flash-lite) в обох прогонах бездоганно виконали вимогу «без markdown» у Завданні 2, повернувши чистий сирий JSON, який одразу розпарсився через json.loads() на 2 бали. Водночас варто зауважити, що локальні моделі всередині валідного JSON-формату підставили вигадані роки відкриття театру (1933/1903 та 1922/1935), а в Завданні 4 («Лише код») хмарна модель все ж таки додала обгортку ````prolog`.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Так, у локальних моделей зафіксовано помітні мовні проблеми: llama3.2:3b у Завданнях 5 і 6 переходила на суржик, російську та англійську мови («Обрахуjte общий витрат», «Окончательный ответ 726 долларов США», «Ціна на-room», «водяна споріднена», «своя фітнес-центр», «більші номера»). Модель qwen3:4b тримала українську мову значно краще, але в Завданні 6 припускалася неточних кальок та граматичних узгоджень («для дешевого вибору», «дешевий тип гостини», «відрізняється від готелі»), і лише хмарна gemini-3.5-flash-lite написала текст бездоганною літературною українською мовою.

**Приклад провалу (крок 7)**

1. Повний текст запиту:

"Поїздка 6 днів: житло 900/добу, їжа 500/день, дорога 2400. Троє, житло й дорога порівну. Скільки заплатить кожен? Покажи хід розв'язання."

2. Повний текст відповіді:

Прогін 1:

"Покажем хід розв'язання:\n\n1. Обрахуjte общий витрат на житло для трьох осіб:\n - 3 х 900 долларів на житло на рік = 2700 долларів на рік\n - 2700 долларов США на рік / 365 днів в році = 7,43 доллара США на день\n\n2. Обрахуjte общий витрат на їжу для трьох осіб:\n - 3 х 500 долларов на їжу на рік = 1500 долларов на рік\n - 1500 долларов США на рік / 365 днів в році = 4,14 доллара США на день\n\n3. Обрахуйте общий витрат на дорогу для трьох осіб:\n - 2400 долларов США на дорогу (паркінг та інші витрати)\n\n4. Обрахуйте общий витрат на добіг за весь рік для трьох осіб:\n - 7,43 доллара США на день х 365 днів в році = 2709 долларов США на рік\n\n5. Подійте третину витрат на житло між трьома особами:\n - 2700 долларов США / 3 = 900 долларов США на рік\n - 900 долларов США / 365 днів в році = 2,47 доллара США на день\n\n6. Подіть третину витрат на дорогу між трьома особами:\n - 2400 долларов США / 3 = 800 долларов США на рік\n - 800 долларов США / 365 днів в році = 2,19 долларов США на день\n\n7. Подіть третину витрат на їжу між трьома особами:\n - 1500 долларов США / 3 = 500 долларов США на рік\n - 500 долларов США / 365 днів в рік = 1,37 доллара США на день\n\n8. Обрахуйте витрат на добіг для трьох осіб за день:\n - 2,47 долларов США на день на житло + 2,19 долларов США на день на дорогу + 1,37 долларов США на день на їжу = 5,99 долларов США на день\n\n9. Обрахуйте витрат на добіг для трьох осіб за рік:\n - 5,99 долларов США на день х 365 днів в році = 2178 долларов США на рік\n\n10. Обрахуйте витрат на добіг для кожного з трьох осіб:\n - 2178 долларов США на рік / 3 особи = 726 долларов США на рік на людину\n\nОкончательный ответ 726 долларов США."

Прогін 2:

"Вказник, де я вам допоможу:\n\n1. Водярня 3 людини: 900/3 = 300/добу\n2. Їж 3 людини: 500*3 = 1500/день\n3. Дорога 3 людини: 2400/3 = 800/добу\n4. Обіг 3 людини: 300+800 = 1100/добу\n5. Обіг 1 людини: 1100/3 = 366,67/добу"

3. Яка правильна відповідь мала бути:

Поїздка триває 6 днів для 3 осіб. Загальна вартість житла за 6 діб становить 900 * 6 = 5400 грн, а оскільки житло ділиться порівну на трьох, кожен платить 5400 / 3 = 1800 грн. Вартість дороги (2400 грн) також ділиться порівну на трьох: 2400 / 3 = 800 грн з кожного. Витрати на їжу становлять 500 грн/день на людину, тобто за 6 днів кожен витратить 500 * 6 = 3000 грн. Сумарно кожен учасник поїздки заплатить: 1800 + 800 + 3000 = 5600 грн. (Навіть при альтернативному трактуванні, якому слідувала модель qwen3:4b, де 500 грн/день на їжу ділиться на трьох, сума мала б скласти 1800 + 800 + 1000 = 3600 грн, але аж ніяк не 726 доларів чи 366,67/добу).

4. Пояснення, чому так сталося:

Мала інструктивна модель llama3.2:3b (3 млрд параметрів) не має прихованої фази попереднього планування (thinking) і змушена одночасно утримувати в увазі багатокомпонентну текстову умову (кількість днів, кількість людей, різні правила розподілу для житла, дороги та їжі) і генерувати розрахунок недостатньо засвоєною українською мовою. Через перевантаження механізму уваги та авторегресивну природу генерації модель у 1-му прогоні цілковито загубила число «6 днів» із промпту, підмінила його шаблонним фінансовим патерном розрахунку річного бюджету («365 днів в році» у «долларах США»), спочатку помножила суми на 3, потім поділила на 365, знову помножила на 365 і двічі поділила на 3, причому в процесі зірвалася на суміш російської, української та польської/чеської («Obrachujte») мов. У 2-му прогоні модель замість множення на 6 днів просто склала добову частку житла (300) із усією дорогою, поділеною на 3 (800), забула додати їжу і вдруге поділила суму на 3, замінивши слово «житло» безглуздим словом «Водярня». Це наочно демонструє, як в авторегресивній моделі без етапу самоперевірки один хибно обраний на старті токен лавиноподібно руйнує всю подальшу логіку розв'язання.

**Відповіді на питання (крок 8)**

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: Окрім ручного оцінювання за рубрикою, де хмарна модель набрала 11 з 12 балів, на практиці широко використовують автоматизовані бенчмарки (наприклад, MMLU, HumanEval). Також розповсюджений підхід «LLM-as-a-Judge», коли потужніша модель (наприклад, GPT-4) автоматично оцінює відповіді менших, та A/B-тестування, що спирається на відгуки реальних користувачів.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: У навчальних масивах даних фрагменти коду та JSON майже завжди обгорнуті в блоки Markdown (```), що створює дуже сильну статистичну звичку. Хоча в наведеному тесті всі три моделі успішно впоралися із завданням «Формат»,

отримавши по 2 бали, на практиці малі моделі часто не мають достатньої місткості уваги, щоб подолати цей завчений шаблон заради однієї заперечної інструкції.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: Навчальні датасети здебільшого складаються з англійських текстів, тому словниковий запас і граматика інших мов представлені набагато слабше. Через малу кількість параметрів локальним моделям бракує ємності для якісного засвоєння української, що підтверджується нульовою оцінкою llama3.2:3b за це завдання, тоді як велика хмарна модель отримала максимальні 2 бали.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: Генерація тексту спирається на ймовірнісний вибір наступного токена, і наявність параметрів випадковості (наприклад, температури) призводить до різних результатів. Ця нестабільність найбільш виражена у слабких моделей: llama3.2:3b змінила свої результати у 5 з 6 завдань між двома прогонами, тоді як хмарна модель не мала жодної розбіжності.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Відповідь: Галюцинація – це генерація мовною моделлю впевненої, але фактично хибної інформації. Модель радше вигадає відповідь, ніж визнає незнання, оскільки її базова архітектура оптимізована на безперервне статистичне передбачення наступного слова, а не на перевірку істинності; наочно це видно з того, що обидві локальні моделі отримали 0 балів у завданні «Факт».

**Висновок за завданням 5**

Експеримент довів, що оцінювання за фіксованою рубрикою з 6 різнопланових завдань і дворазовим прогоном виявилося значно надійнішим та інформативнішим інженерним інструментом, ніж суб'єктивне враження від спілкування з моделлю, оскільки дозволило чітко кількісно розділити можливості моделей: 3/12 у llama3.2:3b, 8/12 у qwen3:4b та 11/12 у gemini-3.5-flash-lite. Найбільш показовими для диференціації моделей стали Завдання 1 (Факт), яке миттєво виявило

галюцинації обох локальних моделей на регіональних українських даних і довело необхідність використання RAG, Завдання 5 (Логіка), яке продемонструвало прірву між інструктивною 3B-моделлю (0 балів), міркувальною 4B-моделлю (1 бал) та хмарною моделлю (2 бали), і Завдання 3 (Стиснення з числами), де qwen3:4b та gemini-3.5-flash-lite ідеально зберегли всі 6 чисел із наданого контексту на противагу llama3.2:3b. Для роботи з українськомовним текстом я однозначно обрав би хмарну модель gemini-3.5-flash-lite, яка єдина забезпечила природну літературну українську мову без русизмів і кальок (2/2 бали), а за суворої вимоги локального виконання – лише qwen3:4b, оскільки llama3.2:3b є непридатною для україномовних задач через постійні зриви на суржик і вигадані конструкції.

### Завдання 6. Керування випадковістю: temperature і top_p

Таблиця 10 – Вплив параметрів на різноманітність відповідей

| Параметр | Значення | Унікальних із 5 (творче) | Правильних із 5 (фактичне) |
|---|---|---|---|
| temperature | 0.0 | 2 | 5 |
| temperature | 0.7 | 5 | — |
| temperature | 1.5 | 5 | 2 |
| top_p | 0.1 | 1 | — |
| top_p | 0.5 | 5 | — |
| top_p | 1.0 | 5 | — |

Таблиця 11 – Перевірка відтворюваності

| Показник | Значення |
|---|---|
| Кількість запусків | 10 |
| temperature | 0.0 |

| | |
|---|---|
| Унікальних відповідей | 1 |
| Усі відповіді однакові (так / ні) | так |

**Відповіді на питання (крок 7)**

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Для агента, що повертає JSON, слід встановити temperature = 0.0, оскільки це вмикає режим жадібного декодування (завжди обирається найімовірніший токен), що забезпечує максимальну стабільність структури, суворе дотримання синтаксису та мінімізує ризик появи випадкових зайвих символів.

2. Яке — генератору варіантів заголовків?

Для генератора варіантів заголовків оптимальним є діапазон temperature = 0.7–0.9, оскільки, як показав експеримент, вже при 0.7 модель видає 5 із 5 унікальних творчих варіантів без втрати граматичної зв'язності та здорового глузду.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Параметр temperature змінює контрастність усього розподілу ймовірностей (сплющує його при високих значеннях, підтягуючи шанси малоймовірних слів), тоді як top_p не змінює співвідношення ймовірностей між лідерами, а жорстко відсікає «хвіст» малоймовірних токенів, залишаючи лише найімовірніше ядро з сумарною ймовірністю p. Саме тому при top_p=0.5 та 1.0 (за фіксованої помірної temperature=0.7) відповіді залишаються різноманітними (5 з 5), але значно структурованішими за списки при temperature=1.5.

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Ні, temperature = 0.0 не гарантує абсолютної стовідсоткової відтворюваності у всіх випадках через апаратну недетермінованість паралельних операцій із рухомою комою на процесорі чи відеокарті. Хоча в цільовому експерименті з 10 запусків короткого фактичного питання модель видала 10 абсолютно ідентичних відповідей (Рік першого польоту людини в космос - 1961 рік., 1 унікальна), на довшому

творчому промпті при тій самій temperature = 0.0 все ж таки проявилася варіативність – отримано 2 унікальні відповіді з 5 (1-й прогін відрізнявся від 2–5 прогонів).

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися.

На фактичному питанні при temperature = 1.5 модель видала суцільну псевдоісторичну галюцинацію: «Первого польоту людини в космос був провів astronaut Джohny Гленна в ході пілотажного політу Space Scout на кораблі «Серінгопос»... у червені 1962-го року космонавт Тито Ван дер Лін (Варвариж) здійснював перше польото в СРСР на радянському кораблю «Восток-3А»... 23 лютого 1961 року Рейкяйн 3 стартував з космодрому Каплін-Звезда в Нерюжском (Польша) в перший польот у космосу з космонавтом Юрієм Гагаріном...» (а в 3-му прогоні взагалі назвала дату «night of October 12, 1964», тоді як у творчому завданні заявила: «Почніючи з серіалу Netflix «Stranger Things», новішої серії сервісі обміну речами набувають гонітогузного характеру»). Така маячня з'явилася тому, що при діленні логітів на високу температуру 1.5 перед функцією softmax різниця між високо- та низькоймовірними токенами нівелюється: модель починає випадково обирати рідкісні слова й уламки складів із хвоста розподілу, а кожен такий хибний токен далі лавиноподібно зсуває авторегресивний контекст у повну втрату змісту.

**Відповіді на питання (крок 8)**

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Відповідь: Математично параметр temperature ділить логіти (сирі передбачення моделі) перед застосуванням функції softmax. Значення менше одиниці роблять розподіл гострішим, виділяючи найімовірніші токени, тоді як вищі значення (наприклад, 1.5) згладжують його, дозволяючи моделі генерувати більш різноманітні, але менш точні відповіді.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Відповідь: На відміну від temperature, що змінює ймовірності абсолютно всіх можливих токенів, top_p (nucleus sampling) повністю відкидає малоймовірні варіанти. Він формує динамічний пул кандидатів, сумарна ймовірність яких дорівнює значенню p, тому при top_p=0.1 генерується лише 1 унікальна відповідь із 5 через сильне звуження вибору.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: Офіційна документація більшості провайдерів (наприклад, OpenAI) рекомендує змінювати лише один із цих параметрів, залишаючи інший за замовчуванням. Оскільки їх одночасне налаштування ускладнює передбачення результату, для досягнення максимальної варіативності достатньо підвищити temperature до 0.7 чи вище, або top_p до 0.5 чи 1.0.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: Алгоритм жадібного декодування при temperature=0 мінімізує випадковість, проте мікроскопічні похибки округлення при паралельних обчисленнях на GPU (недетермінованість операцій з рухомою комою) іноді можуть змінити результат. Хоча в цільовому тесті на відтворюваність при temperature=0.0 за 10 запусків модель показала стовідсоткову стабільність, у творчому завданні незначна варіативність усе ж проявилася (2 унікальні відповіді з 5).

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: Для витягу структурованих даних, генерації коду або точних відповідей використовують temperature=0.0, оскільки це забезпечує найвищу фактичну точність (5 із 5 правильних). Для написання креативних текстів або генерації ідей значення підвищують до 0.7 або вище, що дає максимальну різноманітність (5 унікальних відповідей), але суттєво знижує достовірність фактів (лише 2 з 5 при значенні 1.5).

**Висновок за завданням 6**

Експеримент довів, що параметри temperature і top_p є не «регуляторами розумності» моделі, а прямими математичними регуляторами випадковості вибору наступного токена. При temperature = 0.0 (та top_p = 0.1) модель демонструє максимальну стабільність і дає 5 із 5 правильних відповідей на фактичне питання про рік польоту в космос (1961) та 10 із 10 однакових відповідей у тесті відтворюваності, тоді як підвищення температури до 1.5 призводить до важких галюцинацій із вигаданими космонавтами, кораблями та датами. Для наступних лабораторних робіт, де модель виступатиме як програмний агент і повинна надійно повертати структуровані дані (схеми JSON/Pydantic) та викликати інструменти, я однозначно обиратиму temperature = 0.0 (залишаючи top_p = 1.0 за замовчуванням). Стандартне налаштування «за замовчуванням» (temperature = 0.7 або 1.0), яке використовується у звичайних чат-ботах, є непридатним для програмних агентів, оскільки вносить зайвий стохастичний шум, провокує галюцинації у фактах і підвищує ризик зламати машинозчитуваний формат відповіді.

**Загальні висновки**

Таблиця 12 – Підсумкове порівняння моделей

| Критерій | Локальна мала (3B) | Локальна більша (4B/8B) | Хмарна |
|---|---|---|---|
| Швидкість генерації | Від 10,7 до 115,3 симв./с (48,08 ток./с) | Близько 22,29 ток./с, повільніша через генерацію thinking | Від 75,2 до 776,2 симв./с |
| TTFT | Стабільний: 2,135–2,155 с | Повільніший за 3B-модель, ~3,4–9,5 с разом із завантаженням | Швидкий: 0,724–0,842 с, можливі стрибки до 5–6 с при навантаженні мережі |

| | | | |
|---|---|---|---|
| Якість, балів /12 | 3 / 12 | 8 / 12 | 11 / 12 |
| Стабільність формату (завдання 2) | Стабільно 2 / 2 (чистий JSON) | Стабільно 2 / 2 (чистий JSON) | Стабільно 2 / 2 (чистий JSON, але у завданні 4 додала Markdown-блок) |
| Потрібні ресурси | Мінімальні, ~2 ГБ диска, від 8 ГБ RAM, працює на звичайному процесорі | Середні, ~2,5 ГБ диска, потребує більше RAM/CPU на довгі ланцюжки роздумів | Мінімальні локально, потрібні лише доступ до інтернету та API-ключ |
| Вартість | Безкоштовно | Безкоштовно | Безкоштовний тариф із rate limit, далі – оплата за токени |
| Куди йдуть ваші дані | Залишаються виключно на локальному комп'ютері | Залишаються виключно на локальному комп'ютері | Передаються через інтернет на сервери провайдера (Google) |
| Працює без інтернету | Так | Так | Ні |

| | | | |
|---|---|---|---|
| Коли обирати саме її | Для найпростіших чернеток, перевірки працездатності коду та базового парсингу в умовах повної офлайн-ізоляції та суворої приватності на слабкому ПК | Для локальної обробки конфіденційних даних, які не можна передавати в хмару, коли задача потребує багатокрокової логіки, математики, написання коду або стиснення наданого тексту | Для всіх основних продуктових і клієнтських задач, що вимагають високої швидкості (низького TTFT і високої пропускної здатності), знання реальних фактів, складної логіки та бездоганної української мови за відсутності заборон на передачу даних назовні |

1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;

2) Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;

3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;

4) Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;

5) Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій.